

2. Calculate the distance between a new entry and other existing values using the Euclidean distance formula.

Brightness	Saturation	Class
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

```
In [1]: # Importing Required Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn import metrics
from sklearn.preprocessing import StandardScaler
import seaborn as sn
import matplotlib.pyplot as plt
```

```
In [2]: # Creating the Dataset from the given table
data = pd.DataFrame({
    'Brightness': [40, 50, 60, 10, 70, 60, 25],
    'Saturation': [20, 50, 90, 25, 70, 10, 80],
    'Class': ['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']
})

print(data.shape)
data.head()
```

(7, 3)

```
Out[2]:
```

	Brightness	Saturation	Class
0	40	20	Red
1	50	50	Blue
2	60	90	Blue
3	10	25	Red
4	70	70	Blue

```
In [3]: # Checking for any null values in dataset
data.isnull().sum()
```

```
Out[3]: Brightness    0
        Saturation    0
        Class         0
        dtype: int64
```

```
In [4]: # Encoding the Class Labels
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
data['Class_encoded'] = le.fit_transform(data['Class'])
print('Encoded Classes:', dict(zip(le.classes_, le.transform(le.classes_))))
data
```

Encoded Classes: {'Blue': np.int64(0), 'Red': np.int64(1)}

```
Out[4]:
```

	Brightness	Saturation	Class	Class_encoded
0	40	20	Red	1
1	50	50	Blue	0
2	60	90	Blue	0
3	10	25	Red	1
4	70	70	Blue	0
5	60	10	Red	1
6	25	80	Blue	0

```
In [5]: # Assigning dependent and independent variables
x = data[['Brightness', 'Saturation']]
y = data['Class_encoded']
```

```
In [6]: # Defining a new entry to classify
new_entry = np.array([[45, 55]])
print('New Entry - Brightness: 45, Saturation: 55')
```

New Entry - Brightness: 45, Saturation: 55

```
In [7]: # Calculating Euclidean Distance between the new entry and all existing values
distances = []
for i in range(len(data)):
    dist = np.sqrt((data['Brightness'].iloc[i] - new_entry[0][0])**2 +
                  (data['Saturation'].iloc[i] - new_entry[0][1])**2)
    distances.append(dist)

data['Distance_from_new'] = distances
print('Euclidean Distances from new entry (45, 55):')
data[['Brightness', 'Saturation', 'Class', 'Distance_from_new']]
```

Euclidean Distances from new entry (45, 55):

Out[7]:

	Brightness	Saturation	Class	Distance_from_new
--	------------	------------	-------	-------------------

0	40	20	Red	35.355339
1	50	50	Blue	7.071068
2	60	90	Blue	38.078866
3	10	25	Red	46.097722
4	70	70	Blue	29.154759
5	60	10	Red	47.434165
6	25	80	Blue	32.015621

```
In [8]: # Sorting by distance to find nearest neighbors
sorted_data = data.sort_values('Distance_from_new')
print('Sorted by Distance (Nearest Neighbors):')
sorted_data[['Brightness', 'Saturation', 'Class', 'Distance_from_new']]
```

Sorted by Distance (Nearest Neighbors):

Out[8]:

	Brightness	Saturation	Class	Distance_from_new
--	------------	------------	-------	-------------------

1	50	50	Blue	7.071068
4	70	70	Blue	29.154759
6	25	80	Blue	32.015621
0	40	20	Red	35.355339
2	60	90	Blue	38.078866
3	10	25	Red	46.097722
5	60	10	Red	47.434165

```
In [9]: # Training the Model using KNN (minkowski)
results = []
for i in [1, 2, 3, 4, 5]:
    model = KNeighborsClassifier(n_neighbors=i, metric='minkowski', p=2)
    model.fit(x, y)
    y_pred = model.predict(x)
    Accuracy_score = metrics.accuracy_score(y, y_pred)
    results.append(Accuracy_score)

print('KNN [ minkowski ]')
print('for n_neighbor=3 : ')
model_k3 = KNeighborsClassifier(n_neighbors=3, metric='minkowski', p=2)
model_k3.fit(x, y)
y_pred = model_k3.predict(x)
conf_mat = metrics.confusion_matrix(y, y_pred)
print('\n Confusion Matrix : ', conf_mat)
Accuracy_score = metrics.accuracy_score(y, y_pred)
print('Accuracy Score : ', Accuracy_score)
print('Accuracy in Percentage : ', int(Accuracy_score*100), '%')
print('\n', classification_report(y_pred, y))

print(results)
```

```
KNN [ minkowski ]
for n_neighbor=3 :

Confusion Matrix : [[4 0]
[0 3]]
Accuracy Score : 1.0
Accuracy in Percentage : 100 %
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4
1	1.00	1.00	1.00	3
accuracy			1.00	7
macro avg	1.00	1.00	1.00	7
weighted avg	1.00	1.00	1.00	7

```
[1.0, 1.0, 1.0, 1.0, 1.0]
```

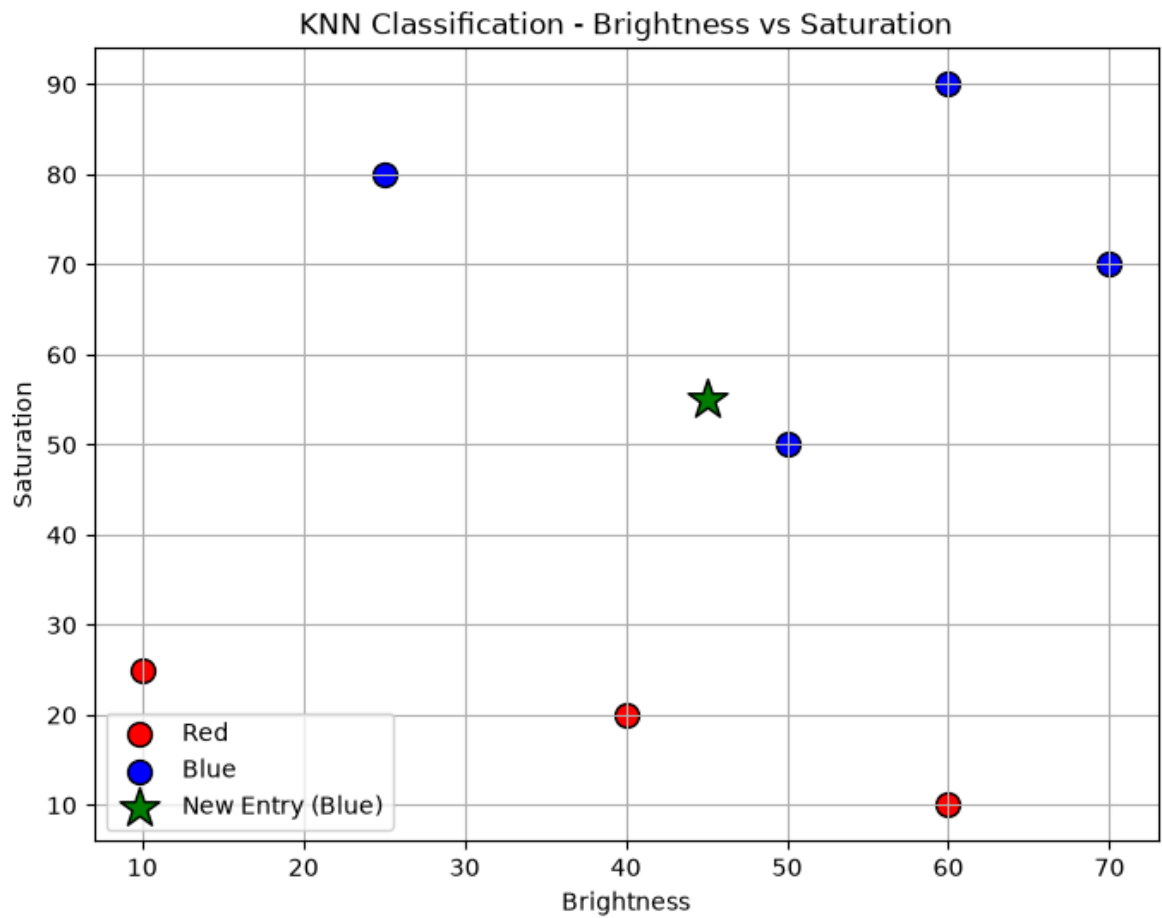
```
In [10]: # Predicting the class of the new entry
prediction = model_k3.predict(new_entry)
predicted_class = le.inverse_transform(prediction)
print(f'Predicted class for new entry (Brightness=45, Saturation=55): {predicted
```

```
Predicted class for new entry (Brightness=45, Saturation=55): Blue
```

```
C:\Users\devad\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn
\utils\validation.py:2827: UserWarning: X does not have valid feature names, but
KNeighborsClassifier was fitted with feature names
warnings.warn(
```

```
In [11]: # Visualization - Scatter plot with new entry
plt.figure(figsize=(8, 6))
colors = {'Red': 'red', 'Blue': 'blue'}
for cls in data['Class'].unique():
    subset = data[data['Class'] == cls]
    plt.scatter(subset['Brightness'], subset['Saturation'],
                c=colors[cls], label=cls, s=100, edgecolors='black')

plt.scatter(new_entry[0][0], new_entry[0][1], c='green', marker='*',
            s=300, label=f'New Entry ({predicted_class[0]})', edgecolors='black')
plt.xlabel('Brightness')
plt.ylabel('Saturation')
plt.title('KNN Classification - Brightness vs Saturation')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [12]: # Comparing accuracy for different values of K
models = pd.DataFrame({
    'n_neighbors': ['1', '2', '3', '4', '5'],
    'Accuracy Score': [results[0], results[1], results[2], results[3], results[4]]
})
models.sort_values(by='Accuracy Score')
print(models.to_string(index=False))
```

n_neighbors	Accuracy Score
1	1.0
2	1.0
3	1.0
4	1.0
5	1.0